

Sandalwood: A High-Performance Python Library for Multivariate Taylor Functions, Differential Algebra, and Autonomous AI Agent Integration

Shashikant Manikonda¹

¹*Independent Researcher*

We introduce **sandalwood** (v0.2.0), an open-source Python library for the creation, manipulation, and composition of Multivariate Taylor Functions (MTF) implementing the mathematical framework of Differential Algebra (DA) and Truncated Power Series Algebra (TPSA). The library provides three primary data types: the real-valued **MultivariateTaylorFunction** (MTF), the complex-valued **ComplexMultivariateTaylorFunction** (CMTF), and the vector-valued **TaylorMap**—the latter being the principal structure for studying coordinate transformations, Hamiltonian flows, and systems of differential equations. A rich calculus layer provides indefinite and definite integration, partial differentiation, Poisson brackets, map composition, and algebraic inversion directly over these objects. Transcendental functions including all standard trigonometric and hyperbolic functions together with their inverses and several specialized physics functions (Gaussian, inverse square root, inverse cube root) are provided as first-class members of the algebra. Performance is achieved through a dual-backend architecture that seamlessly combines a Just-In-Time (JIT) compiled Python/Numba engine for portability with an optional high-performance Fortran backend derived from COSY Infinity. Thread-safe concurrent initialization and an $O(1)$ memory pool enable HPC workflows with full support for parallel batch evaluation. The library also introduces the **agent_tools** module (v0.2.0), a complete integration layer comprising 28 tool definitions that expose the full DA capability set to autonomous AI agents via both the Model Context Protocol (MCP) and LangChain, together with a stateful session registry that prevents LLM context-window saturation. **sandalwood** is ideally suited for applications in nonlinear beam physics, celestial mechanics, electromagnetics, and AI-driven scientific simulation.

I. INTRODUCTION

The analysis of complex dynamical systems often requires accurate representations of nonlinear behaviors that govern long-term stability. While ordinary differential equations (ODEs) can describe these systems, purely analytical solutions are frequently intractable. Traditional numerical integration methods, such as low-order Taylor expansions or Runge-Kutta schemes, compute system states point-by-point in phase space, but they often struggle to efficiently capture the intricate, high-order nonlinear effects required for long-term tracking.

Differential Algebra (DA), computationally realized as Truncated Power Series Algebra (TPSA) [2], offers a profound shift in perspective. Rather than propagating individual points, DA operates on the algebra of functions, treating the truncated Taylor expansion of a function as a single, manipulable mathematical object. Derivatives are propagated algorithmically, avoiding the combinatorial explosion associated with symbolic differentiation. The resulting framework enables the computation of entire families of trajectories from a single calculation.

sandalwood is an open-source Python package that brings high-order DA to the modern scientific computing ecosystem. While established tools like COSY INFINITY [3, 4] offer exceptional Fortran performance, interfacing them with contemporary data science workflows, machine learning frameworks, and automated scientific pipelines presents significant challenges. **sandalwood** addresses this by providing an elegant, object-oriented Python API backed by a sophisticated hybrid execution engine, while remaining fully compatible with both

NumPy [5] and Numba [6].

A second design objective, differentiating **sandalwood** from all prior DA systems, is native support for autonomous AI agents. The **agent_tools** module (introduced in v0.2.0) exposes 28 DA operations as standardized tool calls for large language model (LLM) agents, enabling the creation of fully autonomous scientific computation pipelines without requiring human reformulation of the mathematical problem.

II. THEORETICAL BACKGROUND

A. Truncated Power Series Algebra

Differential algebra is formally concerned with the study of differential equations and operators as algebraic structures, such as differential rings and fields [1]. For computational applications this is realized as TPSA [2]. The core principle is that the local behavior of a sufficiently smooth function can be completely described by its truncated Taylor expansion, referred to as a DA vector.

A DA vector $[f]_n$ represents the Taylor expansion of a function $f : \mathbb{R}^v \rightarrow \mathbb{R}$ at expansion point $\mathbf{x}_0$ up to order n :

$$[f]_n(\mathbf{d}) = \sum_{\substack{|\mathbf{k}|_1 \leq n \\ \mathbf{k} \in \mathbb{N}_0^v}} C_{\mathbf{k}} \cdot d_1^{k_1} \cdot d_2^{k_2} \cdots d_v^{k_v} \quad (1)$$

where v is the number of variables, $\mathbf{d} = (d_1, \dots, d_v)$ represent small deviations from the expansion point, and the

coefficients $C_{\mathbf{k}}$ are the scaled partial derivatives

$$C_{\mathbf{k}} = \frac{1}{\mathbf{k}!} \left. \frac{\partial^{|\mathbf{k}|} f}{\partial x_1^{k_1} \cdots \partial x_v^{k_v}} \right|_{\mathbf{x}_0}. \quad (2)$$

The total number of distinct monomials (and hence storage slots) for a DA vector of order n in v variables is

$$N_{\text{terms}} = \binom{n+v}{v}. \quad (3)$$

For large n and v this grows polynomially, making sparse storage of only non-zero coefficients necessary for practical high-order computation.

B. Taylor Maps and Hamiltonian Flows

A *Taylor map* $\mathbf{M} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a vector-valued function where each component M_i is a DA vector. Taylor maps are the primary computational vehicle for representing the one-turn maps of particle accelerator lattices, the state-transition functions of nonlinear control systems, and the flow maps of Hamiltonian systems.

For a Hamiltonian system with phase space coordinates (q_i, p_i) , the *Poisson bracket* of two observables F and G is

$$\{F, G\} = \sum_{i=1}^n \left(\frac{\partial F}{\partial q_i} \frac{\partial G}{\partial p_i} - \frac{\partial F}{\partial p_i} \frac{\partial G}{\partial q_i} \right), \quad (4)$$

a fundamental operation for studying the evolution of observables and for constructing symplectic integrators. **sandalwood** exposes Poisson brackets as a first-class operation on MTF objects.

III. SOFTWARE ARCHITECTURE

sandalwood is designed around an intuitive object-oriented API that abstracts the underlying computational complexity.

A. Core Data Types

The library exposes three primary data types, all sharing a unified coefficient storage model:

- **MultivariateTaylorFunction (MTF)**: A real-valued scalar DA vector. Coefficients are stored as a pair of parallel NumPy arrays: a 2D integer array `exponents` of shape (N_{terms}, v) containing multi-index exponents, and a 1D float64 array `coeffs` of the corresponding coefficient values. All standard arithmetic operators $(+, -, *, /, **)$ and unary negation are overloaded.

- **ComplexMultivariateTaylorFunction (CMTF)**: A subclass of MTF specialized for complex coefficients (dtype `complex128`). Provides additional operations `conjugate()`, `real_part()`, and `imag_part()` for extracting real and imaginary components as standard MTFs.
- **TaylorMap**: A vector-valued function $\mathbb{R}^n \rightarrow \mathbb{R}^m$ represented as an ordered list of MTF components. Supports element-wise arithmetic $(+, -, *)$, map composition, algebraic inversion, truncation, substitution, and sensitivity analysis.

Both MTF and CMTF support lazy synchronization with the backend: Python-side coefficient arrays are populated on demand from the Fortran backend representation, avoiding redundant copies during pure-backend operation chains.

B. Hybrid Execution Engine

To overcome the overhead commonly associated with Python, **sandalwood** employs a layered hybrid strategy selectable at initialization time:

- **Native Python/Numba Backend** (`implementation="python"`): The default for portable deployments. Coefficient operations are implemented in pure NumPy with optional Numba [6] JIT acceleration. The multiplication kernel (`multiply_dense_parallel`) implements a parallel map-reduce strategy with manual thread-chunk isolation over a precomputed dense multiplication table, enabling safe concurrent writes. A separate SIMD-friendly evaluation kernel (`evaluate_dense_kernel`) processes evaluation points in cache-resident blocks of 256 using precomputed power caches, maximizing L1/L2 cache hit rates. A composition kernel (`compose_dense_kernel`) parallelizes the outer-map substitution across Numba threads.
- **COSY Infinity Fortran Backend** (`implementation="cosy"`): For maximum performance in batch operations and high-order compositions, **sandalwood** dispatches calculations to a compiled Fortran core. Python communicates with the Fortran-allocated static memory pool via a direct memory bridge using `ctypes` and Fortran's `ISO_C_BINDING`, allowing zero-copy data transfers. An $O(1)$ `CosyIndexPool` (object pool pattern) manages Fortran DA variable slots, completely eliminating allocation overhead during computation chains.

Backend selection is automatic: if the Fortran library is found at import time, the COSY backend is used by default; otherwise the Python/Numba backend is activated with a log warning.

C. Thread Safety

All class-level state mutations in `initialize_mtf` are guarded by a module-level reentrant lock (`_INIT_LOCK: threading.RLock`). The Fortran index pool uses an independent `threading.RLock` over its `acquire/release` methods, preventing duplicate-index allocation races when multiple threads perform concurrent computation chains. This allows `sandalwood` to be safely used from multi-threaded LangGraph workflows and multi-worker HPC processes.

IV. MATHEMATICAL OPERATIONS

A. Elementary Functions

The `mtf` and `cmtf` namespace objects expose 23 elementary functions as members of the Taylor algebra. All functions handle non-zero expansion points by factoring out the constant term before applying the zero-centered series, ensuring correctness for arbitrary argument values:

- **Trigonometric:** `sin`, `cos`, `tan`, `arcsin`, `arccos`, `arctan`. The sine and cosine expansions use the addition identities $\sin(C+p) = \sin C \cos p + \cos C \sin p$ and analogously for cosine, with the zero-centered helper series computed from dynamic coefficient generators to arbitrary order.
- **Hyperbolic:** `sinh`, `cosh`, `tanh`, `arctanh`.
- **Exponential & Logarithmic:** `exp` (uses $e^{C+p} = e^C \cdot e^p$), `log` (uses $\log(C+p) = \log C + \log(1+p/C)$).
- **Power:** `sqrt` ($\sqrt{C+p}$ via binomial series), `pow`.
- **Physics Specialties:** `gaussian` (e^{-x^2}), `isqrt` ($1/\sqrt{C+p}$), `inv_cbrt` ($1/(C+p)^{1/3}$), `inv_pow_3_2` ($1/(C+p)^{3/2}$). These are required for Biot-Savart and beam optics applications where the denominator contains powers of the distance squared.

B. Differential Calculus

`sandalwood` provides the full differential calculus layer of a DA system:

- **Partial differentiation:** `derivative(f, var_index)` computes $\partial f / \partial x_i$ by applying the standard monomial differentiation rule $\partial / \partial x_i (C_{\mathbf{k}} \prod x_j^{k_j}) = k_i C_{\mathbf{k}} \prod x_j^{k_j - \delta_{ij}}$ to all terms.
- **Integration:** `integrate(f, var_index, lower, upper)` performs both indefinite and definite integration with respect to x_i . Definite integration over $[a, b]$ evaluates the antiderivative at the bounds and returns the result as a constant MTF.

- **Poisson Bracket:** `compute_poisson_bracket(F, G)` computes $\{F, G\} = \sum_i \partial_q F \partial_p G - \partial_p F \partial_q G$ over the canonical pairs (q_i, p_i) , assuming the standard interleaved-coordinate convention ($x_1 = q_1, x_2 = p_1, x_3 = q_2, x_4 = p_2, \dots$).
- **Composition:** MTF objects support substitution of a set of MTFs into their variables via `f.compose({1: g1, 2: g2, ...})`, computing the composite function $f(g_1(\mathbf{x}), g_2(\mathbf{x}), \dots)$. The COSY backend uses optimized recursive substitution; the Numba backend uses the `compose_dense_kernel`.
- **Truncation:** `f.truncate(order)` lowers the maximum order of any MTF or TaylorMap in-place, discarding all terms of total degree exceeding `order`.

C. TaylorMap Operations

The `TaylorMap` class provides operations essential for nonlinear dynamics:

- **Map Composition:** `M1.compose(M2)` computes $M_1 \circ M_2$, substituting each component of M_2 into the variables of each component of M_1 . The COSY backend marshals the entire map for highly optimized recursive substitution.
- **Algebraic Inversion:** `M.invert()` computes the algebraic inverse M^{-1} of an invertible linear-part map. The algorithm requires the linear part (Jacobian) to be invertible; a check via the Jacobian determinant is performed automatically.
- **Batch Evaluation:** `M.neval(points)` evaluates the map at a 2D array of phase-space points simultaneously, dispatching to OpenMP-parallelized Fortran kernels (COSY) or the Numba `evaluate_dense_kernel` (Python).
- **Sensitivity Analysis:** `M.map_sensitivity(scaling_factors)` scales map coefficients by user-specified factors for parametric sensitivity studies.
- **Trace:** `M.trace()` computes the trace of the linear part (Jacobian) of the map, a key invariant for stability analysis of periodic orbits.
- **Component Access:** Individual components are accessible via `M.components[i]`, and can be added (`add_component`), removed (`remove_component`), or extracted by index (`get_component`).
- **Variable Substitution:** `M.substitute({var_id: value, ...})` partially evaluates the map by fixing one or more variable coordinates to specified values.

- **JSON Serialization:** Both MTF and TaylorMap support `to_json()` and `from_json()` for portable serialization, enabling persistence and network transmission of DA objects.

V. CORE FEATURES AND USAGE

`sandalwood` is designed to be highly accessible. The following code example demonstrates initialization, symbolic variable definition, map construction, composition, and inversion—the fundamental operations of an accelerator lattice tracking study.

```

1 from sandalwood import
   MultivariateTaylorFunction as mtf, TaylorMap
2
3 # Initialize: 4 variables (x, px, y, py), up to
   order 5
4 mtf.initialize_mtf(max_order=5, max_dimension=4)
5
6 # Symbolic variables (1-indexed)
7 x, px, y, py = mtf.var(1), mtf.var(2), mtf.var
   (3), mtf.var(4)
8
9 # Define a non-linear one-turn map (thin-lens
   approximation)
10 # F: (x, px, y, py) -> (x + px, px + 0.1*x**2, y
   + py, py)
11 F = TaylorMap([x + px, px + 0.1 * x**2, y + py,
   py])
12
13 # Define a rotation map G (betatron phase
   advance)
14 import math
15 phi = math.pi / 4 # 45-degree phase advance
16 G = TaylorMap([x * math.cos(phi) + px * math.sin
   (phi),
17               -x * math.sin(phi) + px * math.
   cos(phi), y, py])
18
19 # One-turn map: H = G composed with F
20 H = G.compose(F)
21
22 # Compute the algebraic inverse of H
23 H_inv = H.invert()
24
25 # Validate round-trip: H_inv(H(point)) should
   recover point
26 point = [0.01, 0.001, 0.005, 0.0]
27 H_at_point = H.neval([point])[0]
28 recovered = H_inv.neval([H_at_point.tolist()])
   [0]

```

Listing 1. Map composition and inversion in `sandalwood`

A. Differential Calculus Examples

```

1 from sandalwood import mtf
2 from sandalwood.elementary_functions import
   derivative, integrate
3
4 mtf.initialize_mtf(max_order=4, max_dimension=2)
5 q, p = mtf.var(1), mtf.var(2)
6

```

```

7 # Harmonic oscillator Hamiltonian H = (p**2 + q
   **2) / 2
8 H = (p**2 + q**2) / 2.0
9
10 # Time evolution of q: dq/dt = {q, H} = dH/dp
   dqdt = derivative(H, 2) # dH/dp = p
11
12
13 # Definite integration: integrate q^2 dp from 0
   to 1
14 q_sq = q**2
15 integral_val = integrate(q_sq, var_index=2,
   lower=0.0, upper=1.0)

```

Listing 2. Poisson bracket and definite integration

Map composition ($\mathbf{H} = \mathbf{G} \circ \mathbf{F}$) is computationally demanding as it requires substituting high-order polynomials into one another. `sandalwood` optimizes this through Numba-parallelized dense multiplication tables in the native backend and optimized recursive substitution in the Fortran backend.

VI. AI AGENT INTEGRATION

As scientific discovery becomes increasingly automated, computational tools must serve autonomous AI agents as well as human researchers. `sandalwood`'s `agent_tools` module (v0.2.0) provides a complete integration layer that exposes the library's full capability set via a highly optimized, agent-friendly interface. This module is the first DA system designed from the ground up for LLM agent use.

A. Stateful Session Registry and Context Optimization

A major limitation of utilizing Large Language Models for numerical and symbolic tasks is context-window saturation. A high-order MTF or TaylorMap can contain thousands of floating-point coefficients; serializing these into an LLM conversation history immediately saturates the context window and drives up API costs.

The `agent_tools` module solves this with a *stateful Session Registry* pattern (`registry.py`). When an agent creates, parses, or modifies a DA object, the server registers the object in an in-memory dictionary keyed by a `session_id`, and returns a lightweight JSON reference string (e.g., `{"ref": "mtf_0", "message": "..."}.`). The agent subsequently references these short strings in all future tool calls—composition, differentiation, evaluation, inversion—without ever receiving or transmitting the raw coefficient data. The registry implements the following public API functions:

- `register_object(obj, name, session_id):`
stores an object and returns its reference name.
- `get_object(ref, expected_type, session_id):`
retrieves and type-checks a registered object.

- `list_session_variables(session_id)`: returns a shallow copy of the session’s registered objects.
- `list_all_sessions()`: returns all sessions and their registered variables.
- `prune_registry(session_id)`: clears all objects for a session, releasing memory.

All registry operations are protected by a `threading.Lock`, enabling concurrent agents to operate in isolated sessions without contention.

B. Tool Suite: 28 Operations

The `agent_tools` module exposes 28 operations spanning the complete library capability:

All tools return a standardized `ToolSuccessResponse` / `ToolErrorResponse` JSON envelope. Errors are caught internally and returned as structured JSON with an error code (`INVALID_INPUT`, `COMPUTATION_ERROR`, `REGISTRY_ERROR`, `SYSTEM_ERROR`), enabling the autonomous agent to observe the failure and perform dynamic self-correction within its reasoning loop without crashing the server.

C. Standard Protocols and Tool Wrappers

To achieve compatibility across diverse agentic frameworks, the `agent_tools` module provides two primary interfaces:

- **Model Context Protocol (MCP) Server:** `mcp_server.py`, powered by `FastMCP`, exposes all 28 tools as MCP tool calls and maps the session registry as read-only resources (`registry://variables`, `registry://variable/{name}`), enabling LLMs to enumerate and query registered variables on demand. A specialized instruction template `sandalwood-da-expert` guides the LLM’s reasoning about DA concepts. The `registry://variables` resource enumerates objects across all active sessions.
- **LangChain Tools:** All 28 functions are wrapped with the `LangChain @tool` decorator (`langchain.tools.py`), with `Pydantic v2` input schemas (`schemas.py`) for out-of-the-box integration into `LangGraph` and other agentic graph frameworks.

D. Natural Language Expression Parsing

AI agents frequently struggle with the precise array constructs required by numerical libraries. `sandalwood`

provides a robust parser (`parser.py`) that enables agents to construct MTFs directly from natural mathematical strings (e.g., `"x1**2 + sin(x2)"`). The parser supports all 23 elementary functions in the library’s algebra, variable substitution by name (`x1`, `x2`, ..., `xN`), and automatic dimension consistency checking.

E. Interactive Dashboard

A Gradio [7] web dashboard (`mcp_dashboard/app.py`) connects a Google Gemini LLM agent to the `Sandalwood` MCP server in a side-by-side interface showing the chat history, tool calls, and a live registry inspector panel. The model is configurable via the `SANDALWOOD_GEMINI_MODEL` environment variable.

VII. PERFORMANCE AND OPTIMIZATION

Achieving high performance in Python requires careful management of memory and loop execution. `sandalwood` introduces several critical optimizations:

- **Precomputed Dense Multiplication Tables:** At initialization, a dense multiplication table of shape $(N_{\text{terms}}, N_{\text{terms}})$ is precomputed for each dimension up to `max_dimension`. The table stores the output index of the product of any two monomials, or `-1` if the product exceeds the truncation order. This reduces each coefficient multiplication to a single table lookup, replacing the otherwise expensive binary search over sorted exponent tuples.
- **Numba JIT Compilation with Parallel Map-Reduce:** The `multiply_dense_parallel` kernel partitions the outer operand’s terms into per-thread chunks, accumulates products in thread-local buffers, and reduces at the end—ensuring thread safety without atomic operations. The evaluation kernel `evaluate_dense_kernel` processes points in blocks of 256, pre-computing power caches in a `(dim, max_order+1, block_size)` layout to maximize SIMD vectorization potential.
- **$O(1)$ COSY Index Pool:** The Fortran backend manages DA variable indices through an object pool (`CosyIndexPool`) protected by a reentrant lock. Acquisition and release of Fortran memory slots are $O(1)$ operations, eliminating the scan-for-free-slot overhead of earlier designs.
- **Zero-Copy Fortran Bridge:** Data transfer between Python and the Fortran static memory pool uses direct pointer access via `ctypes` and `ISO_C_BINDING`. This avoids serialization—no arrays are copied during a computation chain that remains entirely within the Fortran backend.

Category	Tools	Count
Setup	<code>initialize_sandalwood</code>	1
Construction	<code>parse_expression_to_mtf</code> , <code>create_taylor_map</code>	2
Evaluation	<code>evaluate_mtf</code> , <code>evaluate_taylor_map</code> , <code>evaluate_mtf_batch</code> , <code>evaluate_taylor_map_batch</code>	4
Calculus	<code>compute_partial_derivative</code> , <code>integrate_mtf</code> , <code>compute_poisson_bracket</code>	3
Arithmetic	<code>perform_mtf_arithmetic</code> , <code>perform_complex_operation</code>	2
Composition	<code>compose_taylor_maps</code> , <code>compose_mtfs</code>	2
Substitution	<code>substitute_variable_in_mtf</code> , <code>substitute_in_taylor_map</code>	2
Manipulation	<code>truncate_object</code> , <code>extract_map_component</code> , <code>get_mtf_coefficient</code>	1
Diagnostics	<code>analyze_taylor_map</code> , <code>invert_taylor_map</code> , <code>compute_map_sensitivity</code> , <code>analyze_mtf_diagnostics</code> , <code>mtf_info</code> , <code>taylor_map_info</code>	5
Session	<code>list_session</code> , <code>clear_session</code>	2

TABLE I. The 28 agent tools grouped by functional category.

- **OpenMP Parallel Batch Evaluation:** Evaluating a TaylorMap at millions of phase-space points is offloaded to an OpenMP-parallelized Fortran kernel via the `neval` interface, yielding speedups of up to $100\times$ compared to sequential NumPy broadcasting.
- **Structure of Arrays (SoA) Layout:** Specialized physics kernels enforce SoA memory layouts, ensuring that all values for the same field component across many source elements are contiguous in memory, maximizing cache locality and SIMD vectorization during batch processing of electromagnetic field integrals.

Benchmark results for large-scale operations (e.g., 100,000-element Biot-Savart integrations) demonstrate that the SoA layout and fast-path JIT dispatch yield speedups exceeding $400\times$ compared to naïve object-oriented Python implementations.

VIII. SOFTWARE QUALITY AND ECOSYSTEM

`sandalwood` follows modern Python packaging and continuous integration best practices:

- **Comprehensive Test Suite:** 389 unit and integration tests (3 skipped) covering all backends, all mathematical operations, thread safety under concurrent initialization, and the full 28-tool agent API. Tests are organized as `pytest` cases with markers distinguishing fast unit tests from slower demo-based integration tests.
- **Static Analysis:** `ruff` enforces PEP 8 compliance and import hygiene; `mypy` enforces type annotations across the core library modules.

- **Documentation:** Sphinx-generated HTML documentation is built automatically on ReadTheDocs, with inline doctest examples for all public functions.
- **Optional Extras:** Dependencies are organized into optional extras (`agents`, `gui`, `accel`, `torch`, `demos`, `test`, `dev`) to minimize the mandatory install footprint for users who do not need GPU acceleration or the agent stack.
- **Availability:** Source code, documentation, and installation instructions are available at <https://github.com/shashi-manikonda/sandalwood>. Documentation is hosted at <https://sandalwood.readthedocs.io>.

IX. CONCLUSION AND FUTURE WORK

`sandalwood` provides a modern, robust, and high-performance Python interface for Multivariate Taylor Functions, Complex Taylor Functions, and Taylor Maps, implementing the full mathematical framework of Differential Algebra. By successfully bridging the ease-of-use of Python with the raw computational power of Numba and Fortran—while simultaneously providing the first DA library with native autonomous AI agent support—it democratizes access to high-order nonlinear analysis methods for both human researchers and AI-driven scientific pipelines.

The library is particularly well-suited for HPC applications in nonlinear beam optics (one-turn maps, normal forms, resonance analysis), electromagnetics (Biot-Savart integration, field quality analysis), celestial mechanics (transfer orbit sensitivity), and Hamiltonian dynamics (Poisson bracket flow). The AI agent integration pathway enables fully autonomous simulation campaigns where an LLM agent can iteratively build, compose, in-

vert, and analyze Taylor maps without human involvement in the mathematical formulation.

Future open-source development will focus on:

- Expanding native GPU acceleration (e.g., via CUDA kernels and JAX [8]);
- Implementing symplectic integrators and Lie transforms natively as agent tools;
- Adding session time-to-live (TTL) and background garbage collection for long-running MCP server deployments;
- Adding OpenAI and Anthropic Claude tool-call adapters alongside the existing LangChain and

MCP interfaces;

- A persistent registry backend (SQLite or Redis) for multi-process agent scenarios.

ACKNOWLEDGMENTS

The authors acknowledge the use of Gemini 3.5 Flash for assistance in writing the Python code, as well as for structuring the initial LaTeX formatting of this document. All generated code and output were verified by the human authors, who take full responsibility for the scientific integrity of the work.

-
- [1] J. F. Ritt, *Differential Algebra* (American Mathematical Society, New York, 1950).
 - [2] M. Berz, “Differential algebraic foundations of nonlinear beam dynamics,” in *Nonlinear and Collective Phenomena in Beam Physics*, AIP Conf. Proc. **496** (1999), pp. 1–42.
 - [3] M. Berz, “COSY INFINITY—A new general-purpose optics code for the design of particle accelerators,” *Particle Accelerators* **24**, 109–124 (1989).
 - [4] K. Makino and M. Berz, “COSY INFINITY Version 9,” in *Proceedings of ICAP 2006* (Chamonix, 2006), p. 229.
 - [5] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature* **585**, 357–362 (2020).
 - [6] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based python JIT compiler,” in *Proc. LLVM Compilation Infrastructure in HPC* (Austin, TX, 2015), pp. 1–6.
 - [7] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, “Gradio: Hassle-free sharing and testing of ML models in the wild,” *arXiv:1906.02569* (2019).
 - [8] J. Bradbury *et al.*, *JAX: Composable transformations of Python+NumPy programs* (2018), software available at <http://github.com/google/jax>.